

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



**Universidad
Gerardo Barrios**

Informe Final de Proyecto Trabajo de Graduación

Modalidad Preespecialización
en la Especialidad de:
Desarrollo de aplicaciones con inteligencia artificial

Nombre del Proyecto:
EDUIA

Presentado por:
Jonathan Roberto Acosta López SMIS099321
Rene Alexander Araujo Soto SMIS060720
Mario Alexander Hernández Quevedo SMIS075121

San Miguel, 23 de agosto de 2026

INDICE

1. RESUMEN DEL PROYECTO.....	1
2. MARCO CONCEPTUAL.....	2
2.1 Inteligencia Artificial Generativa	2
2.2 Modelos de Lenguaje de Gran Escala (LLMs)	3
2.3 Google Gemini y su integración en aplicaciones	4
2.4 Técnica de Prompting y RAG simplificado	4
2.5 Plataformas educativas y sistemas LMS.....	5
2.6 Desarrollo web con Python y Streamlit	6
2.7 API REST con FastAPI	6
2.8 Contenedores Docker y portabilidad	7
2.9 Integración y entrega continua (CI/CD).....	8
2.10 Pruebas de software automatizadas	8
2.11 Despliegue en la nube con Streamlit Cloud	9
3. ANÁLISIS DEL PROBLEMA.....	10
3.1 Contexto de la situación problemática	10
3.2 Definición del problema.....	11
3.3 Usuarios afectados	11
4. PROPUESTA DE SOLUCIÓN	12
4.1 Descripción de la solución	12
4.2 Funcionalidades principales	12
4.3 Arquitectura de la solución	13
4.4 Justificación de las decisiones tecnológicas	14
5. METODOLOGÍA DE DESARROLLO DEL PROYECTO	14
5.1 Enfoque metodológico	14
5.2 Sesión 1 — Diagnóstico y arquitectura inicial	15
5.3 Sesión 2 — API REST con FastAPI.....	15
5.4 Sesión 3 — Pruebas automatizadas y CI/CD.....	16
5.5 Sesión 4 — Despliegue con Docker.....	16
5.6 Sesión 5 — Observabilidad y pruebas de carga	17
5.7 Sesión 6 — Seguridad, release y despliegue público	17
6. RESULTADOS ALCANZADOS	18
6.1 Producto desplegado y accesible públicamente	18
6.2 Suite de pruebas automatizadas.....	19
6.3 Métricas de rendimiento bajo carga	21
6.4 Pipeline CI/CD funcional	27

6.5 Despliegue reproducible con Docker	28
6.6 Arquitectura de IA mejorada	29
7. CONCLUSIONES GENERALES	29
7.1 Aportes del proyecto	29
7.2 Conocimientos adquiridos	30
7.3 Limitaciones y trabajo futuro	31
8. REFERENCIAS	31

1. RESUMEN DEL PROYECTO

EduIA es una plataforma educativa web desarrollada con Python que integra Inteligencia Artificial Generativa para automatizar y personalizar el proceso de enseñanza-aprendizaje. El proyecto fue desarrollado por el Grupo 2 de la Preespecialización en Desarrollo de Aplicaciones con Inteligencia Artificial de la Universidad Gerardo Barrios durante el Módulo 4.

El problema central que aborda EduIA es la carga de trabajo repetitiva que enfrentan los docentes al crear evaluaciones, calificar entregas y proporcionar retroalimentación individual a sus estudiantes. Esta situación se agrava en grupos numerosos donde resulta prácticamente imposible brindar atención personalizada a cada estudiante. De forma paralela, los estudiantes reciben contenido estático sin adaptación a su ritmo o nivel de aprendizaje individual.

La solución propuesta consiste en una plataforma que utiliza el modelo de lenguaje Google Gemini para automatizar tres funciones clave: la generación automática de exámenes a partir del material del curso, la evaluación inteligente del código fuente de los estudiantes con retroalimentación personalizada, y un sistema de chat educativo contextualizado que responde preguntas utilizando el material del módulo como contexto.

La plataforma fue desarrollada siguiendo una metodología incremental a lo largo de seis sesiones. En la primera sesión se realizó el diagnóstico del problema y se definió la arquitectura inicial. En la segunda sesión se desarrolló la API REST con FastAPI. En la tercera sesión se implementaron pruebas automatizadas y un pipeline de CI/CD con GitHub Actions. En la cuarta sesión se contenerizó la aplicación con Docker y se preparó para el despliegue. En la quinta sesión se implementaron mecanismos de observabilidad y se realizaron pruebas de carga con Locust. En la sexta sesión se abordaron aspectos de seguridad, release y rollback.

El producto final se encuentra desplegado públicamente en la URL <https://eduiaiuqb.streamlit.app/>, cuenta con más de 103 pruebas automatizadas que ejecutan de forma continua mediante GitHub Actions, y ha sido probado bajo carga con hasta 200 usuarios simultáneos. El repositorio público del proyecto se encuentra en <https://github.com/Rene29Alexander/Proyecto-eduIA> con la versión v1.0.0 marcada como release final.

Entre los resultados más destacados se encuentran la reducción del tiempo de creación de evaluaciones para el docente, la provisión de retroalimentación inmediata y personalizada al estudiante, y la demostración de que una plataforma educativa con IA puede construirse, probarse y desplegarse de forma reproducible utilizando herramientas modernas de desarrollo de software.

2. MARCO CONCEPTUAL

2.1 Inteligencia Artificial Generativa

La Inteligencia Artificial Generativa es una rama de la inteligencia artificial que se especializa en la creación de contenido nuevo a partir de patrones aprendidos durante un proceso de entrenamiento con grandes volúmenes de datos. A diferencia de los sistemas de IA tradicionales que clasifican o predicen valores discretos, los modelos generativos producen texto, imágenes, audio o código que no existían previamente en sus datos de entrenamiento.

Los modelos generativos de texto, conocidos como modelos de lenguaje, aprenden la distribución estadística de secuencias de palabras en enormes corpus de texto. Una vez entrenados, son capaces de generar respuestas coherentes, redactar documentos, explicar conceptos, evaluar código y mantener conversaciones contextualizadas. Esta capacidad los hace especialmente útiles en aplicaciones educativas donde la generación de contenido personalizado es un requisito central.

En el contexto educativo, la IA generativa representa un cambio de paradigma. Históricamente, la creación de material de evaluación, la retroalimentación de código y la tutoría personalizada requerían la intervención directa de un experto humano. Los modelos generativos modernos permiten automatizar estas tareas manteniendo un nivel de calidad aceptable para entornos académicos, lo que libera tiempo docente para actividades de mayor valor pedagógico.

2.2 Modelos de Lenguaje de Gran Escala (LLMs)

Los Modelos de Lenguaje de Gran Escala, conocidos por su sigla en inglés LLM (Large Language Models), son redes neuronales de arquitectura Transformer entrenadas con cantidades masivas de texto proveniente de diversas fuentes. Su característica definitoria es la escala: cuentan con miles de millones de parámetros ajustables que les permiten capturar relaciones semánticas y sintácticas complejas del lenguaje natural.

La arquitectura Transformer, introducida en 2017, utiliza un mecanismo de atención que permite al modelo considerar el contexto completo de una secuencia de entrada al generar cada token de salida. Este mecanismo es la base de la capacidad de los LLMs para mantener coherencia en textos largos, seguir instrucciones complejas y adaptar su estilo de respuesta según el contexto proporcionado.

Los LLMs modernos son entrenados en dos fases principales. En la fase de preentrenamiento, el modelo aprende a predecir el siguiente token en secuencias de texto sin supervisión explícita. En la fase de ajuste fino mediante retroalimentación humana (RLHF), el modelo es optimizado para seguir instrucciones y generar respuestas útiles, inofensivas y honestas. Esta segunda fase es la que convierte un modelo de lenguaje en un asistente capaz de responder preguntas educativas con precisión.

2.3 Google Gemini y su integración en aplicaciones

Google Gemini es la familia de modelos de lenguaje multimodal desarrollada por Google DeepMind. Fue diseñada desde su origen para procesar y generar texto, código, imágenes y audio de forma integrada. La variante utilizada en EdulA es gemini-2.0-flash y sus modelos de respaldo, accesibles a través de la API de Google AI Studio.

La integración de Gemini en aplicaciones Python se realiza mediante la librería `google-generativeai`, que proporciona una interfaz de alto nivel para enviar prompts y recibir respuestas del modelo. La comunicación se realiza a través de HTTPS enviando el prompt como texto y recibiendo la respuesta generada, lo que permite incorporar capacidades de IA en cualquier aplicación web sin necesidad de infraestructura de entrenamiento propia.

Una característica relevante de Gemini para aplicaciones educativas es su capacidad para seguir instrucciones estructuradas en el prompt. Esto permite definir el rol del modelo como tutor educativo, establecer restricciones sobre el tipo de respuesta esperada y proporcionar contexto específico del curso que el modelo utilizará como base de conocimiento para sus respuestas.

2.4 Técnica de Prompting y RAG simplificado

El prompting es la técnica de diseñar instrucciones de entrada para guiar el comportamiento de un modelo de lenguaje hacia el resultado deseado. Un prompt bien diseñado incluye el rol que debe asumir el modelo, el contexto relevante, las restricciones de la tarea y el formato esperado de la respuesta. La calidad del prompt determina en gran medida la utilidad y precisión de la respuesta generada.

La técnica de Generación Aumentada por Recuperación, conocida como RAG

(Retrieval-Augmented Generation), combina la capacidad generativa de los LLMs con la recuperación de información relevante de una base de conocimiento externa. En lugar de depender únicamente del conocimiento interno del modelo, el sistema recupera documentos o fragmentos relevantes y los incluye en el prompt para que el modelo base su respuesta en información verificable y actualizada.

EduIA implementa una versión simplificada de RAG: el docente configura el material del módulo (texto o PDF) y la plataforma lo incluye directamente en el prompt enviado a Gemini. De esta forma, el chat educativo responde preguntas basándose en el material específico del curso y no en conocimiento genérico del modelo, lo que aumenta la precisión y relevancia pedagógica de las respuestas.

2.5 Plataformas educativas y sistemas LMS

Un Sistema de Gestión del Aprendizaje, conocido por su sigla en inglés LMS (Learning Management System), es una plataforma de software que centraliza la creación, distribución y seguimiento de contenido educativo. Los LMS tradicionales como Moodle, Canvas o Blackboard ofrecen herramientas para gestionar cursos, módulos, tareas, exámenes y calificaciones, pero generalmente carecen de capacidades adaptativas basadas en IA.

La integración de IA generativa en plataformas educativas representa una evolución significativa respecto a los LMS tradicionales. Las plataformas con IA pueden generar contenido dinámico, adaptar la dificultad de los ejercicios al nivel del estudiante, proporcionar retroalimentación inmediata y personalizada, y detectar patrones de aprendizaje que serían imposibles de identificar manualmente en grupos numerosos.

EduIA se posiciona como un LMS de nueva generación que incorpora IA en tres puntos críticos del proceso educativo: la creación de evaluaciones, la retroalimentación de código y la tutoría conversacional. Este enfoque permite que los docentes se

concentren en la facilitación del aprendizaje mientras la plataforma automatiza las tareas repetitivas de evaluación y retroalimentación.

2.6 Desarrollo web con Python y Streamlit

Python es el lenguaje de programación más utilizado en el campo de la inteligencia artificial y la ciencia de datos debido a su sintaxis clara, su extenso ecosistema de librerías especializadas y su facilidad de integración con APIs externas. Para el desarrollo de aplicaciones web con IA, Python ofrece frameworks que permiten construir interfaces funcionales con un mínimo de código de presentación.

Streamlit es un framework de código abierto que permite convertir scripts de Python en aplicaciones web interactivas sin necesidad de conocimientos de HTML, CSS o JavaScript. Su modelo de ejecución reactivo vuelve a ejecutar el script completo cada vez que el usuario interactúa con la interfaz, lo que simplifica enormemente el desarrollo de aplicaciones de demostración y prototipos funcionales.

En EduIA, Streamlit sirve como la capa de presentación completa de la plataforma. Gestiona la autenticación de usuarios por roles, renderiza los paneles diferenciados para administrador, docente y estudiante, y conecta la interfaz con la lógica de negocio implementada en los módulos de Python. Esta elección permite un desarrollo ágil aunque introduce limitaciones de escalabilidad en producción que se documentan como parte de la deuda técnica del proyecto.

2.7 API REST con FastAPI

Una API REST (Representational State Transfer) es un estilo de arquitectura para servicios web que define cómo los componentes de un sistema distribuido se comunican a través del protocolo HTTP. Las APIs REST utilizan los métodos HTTP (GET, POST, PUT, DELETE) para representar operaciones sobre recursos, y el

formato JSON como estándar de intercambio de datos.

FastAPI es un framework moderno de Python para construir APIs REST de alto rendimiento. Utiliza las anotaciones de tipo de Python y el estándar OpenAPI para generar automáticamente documentación interactiva (Swagger UI), validar los datos de entrada mediante modelos Pydantic y proporcionar respuestas de error estructuradas y consistentes.

En EduIA, FastAPI expone las capacidades de IA del proyecto como servicios independientes accesibles desde cualquier cliente HTTP. Esta separación entre el frontend Streamlit y el backend FastAPI sigue el principio de separación de responsabilidades y permite que los endpoints de IA sean consumidos por otras aplicaciones o testados de forma independiente sin necesidad de interactuar con la interfaz de usuario.

2.8 Contenedores Docker y portabilidad

Docker es una plataforma de virtualización a nivel de sistema operativo que permite empaquetar una aplicación junto con todas sus dependencias en una unidad llamada contenedor. A diferencia de las máquinas virtuales tradicionales, los contenedores comparten el kernel del sistema operativo anfitrión, lo que los hace significativamente más ligeros y rápidos de iniciar.

La portabilidad es la ventaja principal de Docker: un contenedor construido en cualquier máquina funcionará de manera idéntica en cualquier otra máquina que tenga Docker instalado, eliminando el clásico problema de las dependencias de entorno. Esta característica es fundamental en equipos de desarrollo distribuidos y en procesos de despliegue continuo donde la reproducibilidad del entorno es crítica.

El Dockerfile de EduIA utiliza la imagen base python:3.11-slim para minimizar el

tamaño de la imagen, instala únicamente las dependencias necesarias para la API, copia los archivos fuente relevantes y configura un HEALTHCHECK que verifica periódicamente que el endpoint /health responda correctamente. Esta configuración permite desplegar la API de EduIA en cualquier infraestructura que soporte Docker con un solo comando.

2.9 Integración y entrega continua (CI/CD)

La Integración Continua (CI) es una práctica de desarrollo de software en la que los cambios de código son integrados frecuentemente al repositorio principal y verificados automáticamente mediante una suite de pruebas. El objetivo es detectar errores de integración lo antes posible en el ciclo de desarrollo, reduciendo el costo de corrección.

La Entrega Continua (CD) extiende la CI para automatizar el proceso de preparación y despliegue de nuevas versiones del software. En un pipeline CI/CD completo, cada commit que pasa las pruebas automáticas puede ser desplegado a producción sin intervención manual, garantizando que el software en producción siempre corresponda a una versión verificada del código fuente.

EduIA implementa CI mediante GitHub Actions, el servicio de automatización integrado en GitHub. El workflow configurado en ``.github/workflows/ci.yml`` se activa automáticamente en cada push a la rama principal y ejecuta la instalación de dependencias, la verificación de sintaxis del código y la suite completa de pruebas unitarias. Esto garantiza que ningún cambio rompa las funcionalidades existentes sin que el equipo sea notificado inmediatamente.

2.10 Pruebas de software automatizadas

Las pruebas de software son actividades sistemáticas para verificar que un sistema funciona según lo especificado y detectar defectos antes de que lleguen a producción.

Las pruebas automatizadas ejecutan estas verificaciones mediante código, lo que permite repetirlas de forma rápida y consistente en cada cambio del sistema.

Las pruebas unitarias verifican el comportamiento de unidades individuales de código (funciones, clases o métodos) de forma aislada, utilizando datos de entrada controlados y verificando que las salidas correspondan a los valores esperados. Las pruebas de integración verifican que varios componentes del sistema funcionen correctamente al interactuar entre sí.

EduIA cuenta con más de 103 pruebas automatizadas organizadas en cinco archivos que cubren la API REST, el analizador de sintaxis, el analizador de lógica, el sistema de calificación y los gestores del sistema de engagement. Adicionalmente, el archivo `locustfile.py` implementa pruebas de carga que simulan hasta 200 usuarios simultáneos interactuando con los endpoints de la API, proporcionando métricas de rendimiento como tiempo de respuesta percentil 50, percentil 95 y tasa de error bajo carga.

2.11 Despliegue en la nube con Streamlit Cloud

Streamlit Cloud es el servicio de despliegue administrado de Streamlit Inc. que permite publicar aplicaciones Streamlit de forma gratuita directamente desde un repositorio de GitHub. La plataforma detecta automáticamente los cambios en la rama configurada y redespiega la aplicación sin intervención manual, implementando una forma básica de entrega continua.

La gestión de secretos en Streamlit Cloud se realiza a través de un panel de configuración que almacena las variables de entorno de forma segura y las inyecta en la aplicación sin exponerlas en el código fuente ni en el repositorio. Esto permite manejar credenciales como la API key de Gemini de forma segura en un entorno de producción público.

EduIA utiliza Streamlit Cloud como plataforma de despliegue principal para el prototipo académico. La rama `migracion-base-de-datos` del repositorio está conectada directamente al servicio, lo que permite que los cambios aprobados se reflejen en la URL pública <https://eduiaiuqb.streamlit.app/> de forma automática tras cada push a esa rama.

3. ANÁLISIS DEL PROBLEMA

3.1 Contexto de la situación problemática

El sistema educativo enfrenta un desafío estructural relacionado con la escalabilidad de la atención personalizada. En entornos de educación formal con grupos numerosos, los docentes deben distribuir su tiempo y atención entre múltiples estudiantes que avanzan a ritmos diferentes y tienen necesidades de aprendizaje distintas. Esta limitación se manifiesta de forma especialmente aguda en asignaturas técnicas como programación, donde la retroalimentación individualizada sobre el código de cada estudiante requiere una revisión detallada que consume horas de trabajo docente.

El problema se puede desglosar en dos dimensiones principales. Por el lado del docente, la creación de evaluaciones desde cero para cada unidad temática, la calificación manual de ejercicios de programación y la provisión de retroalimentación escrita individualizada consumen una proporción significativa del tiempo de trabajo, dejando menos tiempo disponible para la preparación de clases, la actualización profesional y la atención a estudiantes con dificultades específicas.

Por el lado del estudiante, la educación tradicional ofrece un ritmo uniforme de avance que no se adapta a las diferencias individuales en la velocidad de comprensión y asimilación de conceptos. Un estudiante que comprende rápidamente los conceptos básicos debe esperar al grupo para avanzar, mientras que un estudiante con

dificultades en un tema específico recibe la misma cantidad de práctica que los demás, independientemente de si ha consolidado los fundamentos necesarios.

3.2 Definición del problema

El problema central que EduIA aborda puede formularse de la siguiente manera: los docentes de programación en grupos numerosos no cuentan con herramientas que les permitan escalar la generación de evaluaciones y la provisión de retroalimentación personalizada, mientras que los estudiantes carecen de acceso a tutoría adaptativa que ajuste el ritmo y el contenido a su nivel individual de aprendizaje.

Este problema tiene consecuencias medibles en la calidad del proceso educativo. La retroalimentación tardía o genérica reduce la efectividad del aprendizaje por error, que es uno de los mecanismos más potentes de consolidación de conocimiento en programación. La falta de práctica adaptativa al nivel del estudiante genera brechas de conocimiento que se acumulan a lo largo del curso y dificultan la comprensión de conceptos más avanzados.

3.3 Usuarios afectados

Los principales grupos afectados por el problema son:

Docentes de programación: Profesionales responsables de enseñar lenguajes de programación y lógica computacional en instituciones educativas. Su necesidad principal es reducir el tiempo dedicado a tareas repetitivas de evaluación y retroalimentación para poder enfocarse en la facilitación activa del aprendizaje.

Estudiantes de programación: Personas en proceso de aprendizaje de uno o más lenguajes de programación. Su necesidad principal es recibir retroalimentación

inmediata sobre sus errores y acceder a material de práctica adaptado a su nivel actual de competencia.

Administradores educativos: Responsables de la gestión de la plataforma y del seguimiento del progreso de estudiantes y docentes. Su necesidad principal es contar con una herramienta centralizada que facilite la administración del proceso educativo.

4. PROPUESTA DE SOLUCIÓN

4.1 Descripción de la solución

EduIA es una plataforma web educativa construida con Python y Streamlit que integra el modelo de lenguaje Google Gemini para automatizar y personalizar tres aspectos críticos del proceso de enseñanza-aprendizaje en programación.

La solución se estructura en tres capas: una capa de presentación construida con Streamlit que gestiona la interfaz diferenciada por roles, una capa de lógica de negocio implementada en Python que coordina las operaciones de la plataforma, y una capa de datos basada en SQLite que almacena la información de usuarios, cursos, evaluaciones y resultados.

4.2 Funcionalidades principales

Para el docente:

- Creación y gestión de cursos, módulos y materiales de estudio
- Generación automática de exámenes mediante IA a partir del material del módulo
- Configuración del chat educativo por módulo con material de contexto personalizado
- Revisión de entregas y seguimiento del progreso estudiantil

Para el estudiante:

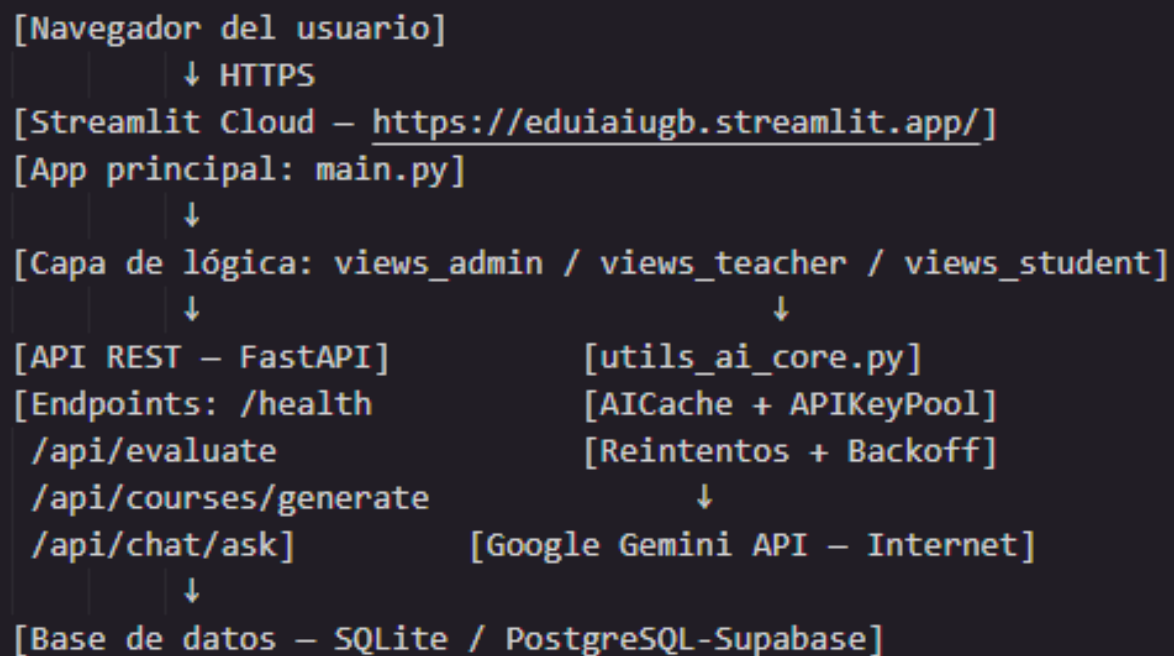
- Acceso a cursos, módulos y materiales
- Entrega de tareas y realización de exámenes
- Chat educativo por módulo donde la IA responde usando el material del curso como contexto
- Academia Personal de Programación: evaluación diagnóstica del nivel, plan de aprendizaje adaptado y ejercicios personalizados con retroalimentación de código en tiempo real

Para el administrador:

- Gestión de usuarios por roles
- Supervisión general de la plataforma
- Gestión de notificaciones del sistema

4.3 Arquitectura de la solución

La arquitectura final de EduIA después de seis sesiones de desarrollo es la siguiente:



4.4 Justificación de las decisiones tecnológicas

Python y Streamlit: Permiten desarrollo ágil de prototipos funcionales con mínimo código de presentación, ideal para un contexto académico con tiempo limitado.

Google Gemini: Ofrece un free tier generoso (1,500 requests/día) sin requerir tarjeta de crédito, con capacidades de generación de texto suficientes para los casos de uso educativos definidos.

FastAPI: Genera documentación automática (Swagger UI), valida entradas con Pydantic y facilita el testing sin necesidad de servidor externo.

SQLite con migración a PostgreSQL: SQLite permite desarrollo sin configuración de servidor. La arquitectura dual implementada permite migrar a PostgreSQL/Supabase para producción sin cambiar la lógica de negocio.

GitHub Actions: Integrado en el repositorio, sin costo adicional, suficiente para automatizar las pruebas del proyecto.

Streamlit Cloud: Despliegue gratuito conectado directamente al repositorio GitHub, sin necesidad de configurar infraestructura de servidor.

5. METODOLOGÍA DE DESARROLLO DEL PROYECTO

5.1 Enfoque metodológico

El proyecto fue desarrollado siguiendo una metodología incremental estructurada en seis sesiones, donde cada sesión añadió una capa de funcionalidad y calidad al producto. Este enfoque permitió gestionar la complejidad del proyecto de forma gradual y contar con un producto demostrable al final de cada iteración.

5.2 Sesión 1 — Diagnóstico y arquitectura inicial

En la primera sesión se realizó el diagnóstico del problema educativo y se definió la arquitectura inicial de la plataforma. Se creó el repositorio en GitHub, se documentó el problema, los usuarios y la propuesta de valor inicial, y se identificaron los riesgos técnicos y la deuda técnica del proyecto.

Los entregables de esta sesión fueron: el archivo README.md con la descripción del proyecto, el documento de arquitectura actual (`docs/arquitectura-actual.md`), el documento de arquitectura objetivo (`docs/arquitectura-objetivo.md`) y el registro de riesgos técnicos (`docs/riesgos-tecnicos.md`).

5.3 Sesión 2 — API REST con FastAPI

En la segunda sesión se separó la lógica de IA del frontend Streamlit mediante la creación de una API REST con FastAPI. Se implementaron los cinco endpoints principales del sistema: GET /health, GET /metadata, POST /api/evaluate, POST /api/courses/generate y POST /api/chat/ask.

Cada endpoint fue diseñado con modelos Pydantic para la validación de entrada, manejo explícito de errores con códigos HTTP apropiados (422 para errores de validación, 503 cuando Gemini no está disponible) y documentación automática generada por FastAPI.

```
```python
Ejemplo del endpoint de evaluación de código
@app.post("/api/evaluate", response_model=EvaluationResponse)
async def evaluate_code(request: EvaluationRequest):
 language = request.language.lower()
 if language not in SUPPORTED_LANGUAGES:
 raise HTTPException(status_code=422, detail=f"Lenguaje no soportado: {language}")
 result = ai_manager.evaluate_code(request.code, language, request.criteria)
 return EvaluationResponse(**result)
```
```

5.4 Sesión 3 — Pruebas automatizadas y CI/CD

En la tercera sesión se implementó la suite de pruebas automatizadas y el pipeline de CI/CD. Se crearon 103 pruebas unitarias distribuidas en cinco archivos que cubren los componentes críticos del sistema sin requerir un servidor corriendo ni una API key real de Gemini.

El pipeline CI/CD fue configurado en ``.github/workflows/ci.yml`` para ejecutarse automáticamente en cada push a la rama principal, verificando que todas las pruebas pasen antes de integrar los cambios.

5.5 Sesión 4 — Despliegue con Docker

En la cuarta sesión se contenerizó la API REST con Docker. Se creó el Dockerfile usando la imagen base `python:3.11-slim`, se configuró el `.dockerignore` para excluir archivos innecesarios y sensibles, y se documentó el plan de infraestructura y los costos operativos.

```
```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements-test.txt .
RUN pip install --no-cache-dir -r requirements-test.txt
COPY api.py config.py database.py utils_ai.py .
COPY models/ ./models/
COPY evaluacion/ ./evaluacion/
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s CMD python -c \
 "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```
```

5.6 Sesión 5 — Observabilidad y pruebas de carga

En la quinta sesión se implementaron mecanismos de observabilidad y rendimiento. Se creó el módulo `utils_ai_core.py` con tres componentes principales: `AI_CACHE` para almacenar en caché las respuestas de Gemini con TTL de 24 horas, `APIKeyPool` para rotar automáticamente entre múltiples API keys cuando se agota la cuota, y `AIManagerCore` con reintentos automáticos con backoff exponencial.

Se implementó el archivo `locustfile.py` con cinco tipos de usuario simulado y se ejecutaron pruebas de carga con 20 y 200 usuarios simultáneos, identificando el cuello de botella principal (latencia de Gemini API) y documentando los resultados en reportes HTML.

5.7 Sesión 6 — Seguridad, release y despliegue público

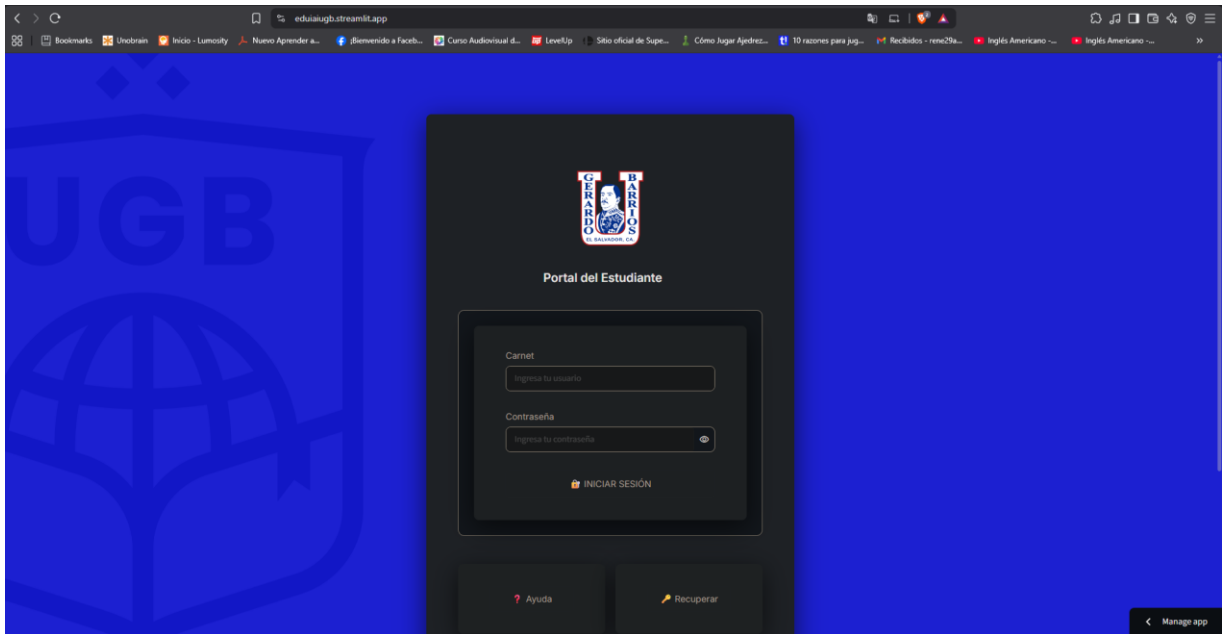
En la sexta sesión se completó el despliegue público en Streamlit Cloud, se realizó la migración de compatibilidad con PostgreSQL/Supabase para producción real, se implementó diseño responsive para dispositivos móviles y se creó el release final v1.0.0 con el manifiesto de release y el plan de contingencia para la demostración.

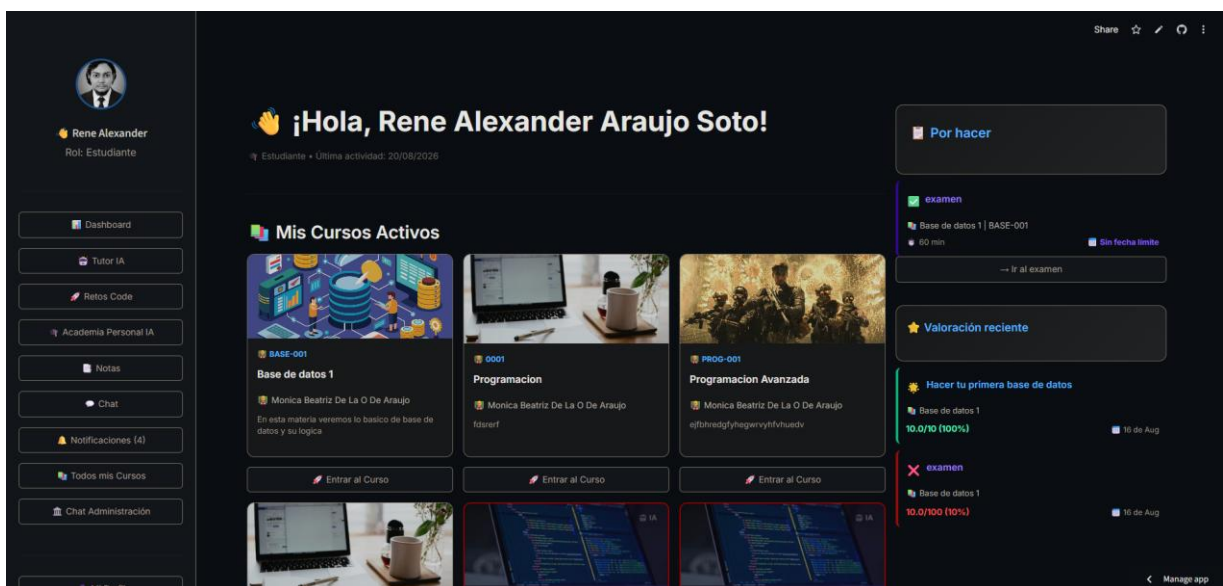
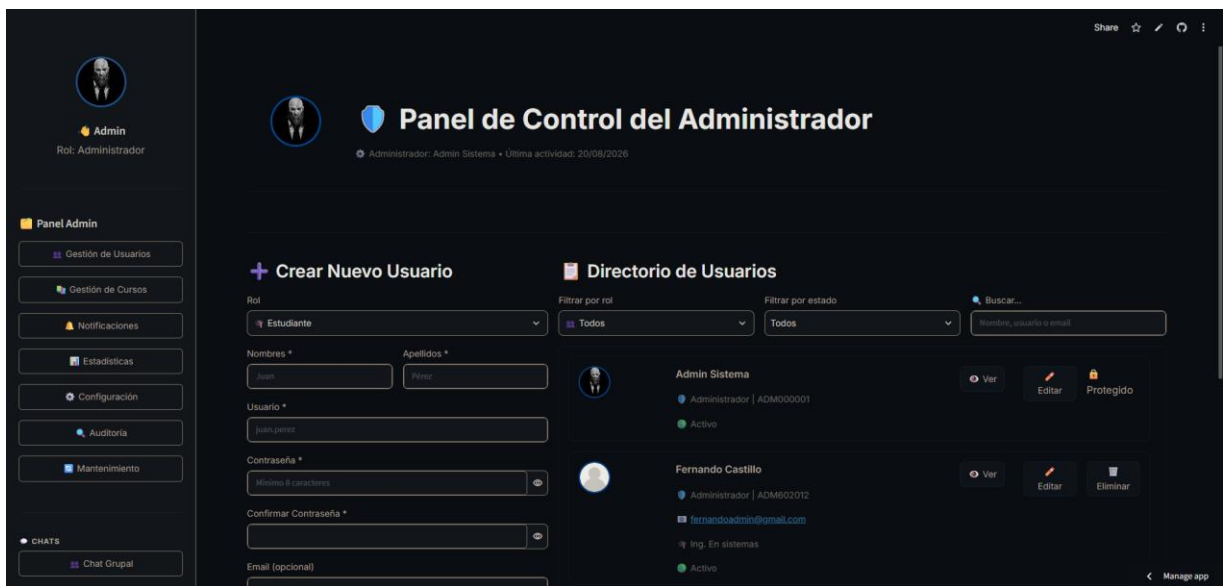
Adicionalmente se implementaron cinco mejoras técnicas: security headers HTTP en todas las respuestas (`X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, `Strict-Transport-Security`, `Referrer-Policy`, `Permissions-Policy`), logging a archivo rotativo en `logs/api.log` con `RotatingFileHandler` de 10 MB y 5 backups, un agente supervisor en background (`agent_monitor.py`) que monitorea logs cada 30 segundos y genera diagnósticos en lenguaje natural con Gemini cuando detecta anomalías, notificaciones automáticas a Discord via webhook cuando se detectan errores o al iniciar la API, y la migración del núcleo de IA de `google-generativeai` a `google.genai` con el nuevo patrón `genai.Client`.

6. RESULTADOS ALCANZADOS

6.1 Producto desplegado y accesible públicamente

El resultado principal del proyecto es la plataforma EduIA disponible públicamente en <https://eduiaiuqb.streamlit.app/>. La plataforma funciona desde cualquier navegador web sin necesidad de instalación, soporta los tres roles de usuario (administrador, docente y estudiante) y tiene conectividad con la API de Google Gemini para las funciones de IA.





6.2 Suite de pruebas automatizadas

Se alcanzaron 103 pruebas automatizadas distribuidas en cinco archivos de prueba. Todas las pruebas pasan correctamente en el entorno local y en el pipeline de GitHub Actions, garantizando la calidad del código en cada integración.

| Archivo de prueba | Tipo | Pruebas |

|---|---|---

| test_api_unit.py | Unitaria | 29 |

| test_analizador_sintaxis.py | Unitaria | 26 |

| test_analizador_logica.py | Unitaria | 18 |

| test_sistema_evaluacion.py | Unitaria | 30 |

| test_engagement_managers.py | Unitaria | ~545 líneas de prueba |

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\Users\rene2\OneDrive\Escritorio\Grupo-2-proyecto> python -m pytest tests/test_api_unit.py tests/test_analizador_sintaxis.py tests/test_analizador_logica.py tests/test_engagement_managers.py -v 2>&1
tests/test_engagement_managers.py::TestPointsManagerLogicaPura::test_coins_earned_menos_de_10_puntos PASSED [ 73%]
tests/test_engagement_managers.py::TestPointsManagerConBD::test_get_user_points_info_sin_registro PASSED [ 74%]
tests/test_engagement_managers.py::TestPointsManagerConBD::test_get_user_points_info_con_registro PASSED [ 75%]
tests/test_engagement_managers.py::TestPointsManagerConBD::test_get_user_coins_sin_registro PASSED [ 76%]
tests/test_engagement_managers.py::TestPointsManagerConBD::test_get_user_coins_con_saldo PASSED [ 77%]
tests/test_engagement_managers.py::TestPointsManagerConBD::test_spend_coins_saldo_insuficiente PASSED [ 78%]
tests/test_engagement_managers.py::TestPointsManagerConBD::test_spend_coins_exitoso PASSED [ 79%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_today_retorna_date PASSED [ 80%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_parse_date_formato_corto PASSED [ 81%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_parse_date_formato_largo PASSED [ 81%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_parse_date_formato_con_microsegundos PASSED [ 82%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_parse_date_none PASSED [ 83%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_parse_date_string_vacio PASSED [ 84%]
tests/test_engagement_managers.py::TestStreakManagerLogicaPura::test_parse_date_formato_invalido PASSED [ 85%]
tests/test_engagement_managers.py::TestStreakManagerConBD::test_get_streak_info_usuario_sin_registro PASSED [ 86%]
tests/test_engagement_managers.py::TestStreakManagerConBD::test_get_streak_info_racha_activa_hoy PASSED [ 87%]
tests/test_engagement_managers.py::TestStreakManagerConBD::test_get_streak_info_racha_en_riesgo PASSED [ 88%]
tests/test_engagement_managers.py::TestStreakManagerConBD::test_use_freeze_sin_congeladores PASSED [ 89%]
tests/test_engagement_managers.py::TestStreakManagerConBD::test_use_freeze_exitoso PASSED [ 90%]
tests/test_engagement_managers.py::TestStreakManagerConBD::test_check_reset_racha_cero_no_resetea PASSED [ 90%]
tests/test_engagement_managers.py::TestChallengeManagerConBD::test_get_user_challenge_status_sin_intentos PASSED [ 91%]
tests/test_engagement_managers.py::TestChallengeManagerConBD::test_get_user_challenge_status_con_intento_fallido PASSED [ 92%]
tests/test_engagement_managers.py::TestChallengeManagerConBD::test_get_user_challenge_status_completado PASSED [ 93%]
tests/test_engagement_managers.py::TestChallengeManagerConBD::test_get_challenge_leaderboard_vacio PASSED [ 94%]
tests/test_engagement_managers.py::TestChallengeManagerConBD::test_get_challenge_leaderboard_con_datos PASSED [ 95%]
tests/test_engagement_managers.py::TestLeaderboardManagerConBD::test_get_global_leaderboard_vacio PASSED [ 96%]
tests/test_engagement_managers.py::TestLeaderboardManagerConBD::test_get_global_leaderboard_all_time PASSED [ 97%]
tests/test_engagement_managers.py::TestLeaderboardManagerConBD::test_get_global_leaderboard_weekly PASSED [ 98%]
tests/test_engagement_managers.py::TestLeaderboardManagerConBD::test_get_user_rank_retorna_entero PASSED [ 99%]
tests/test_engagement_managers.py::TestLeaderboardManagerConBD::test_get_user_rank_primer_en_ranking PASSED [100%]

===== warnings summary =====
tests/test_api_unit.py::TestEvaluateValidaciones::test_evaluate_language_case_insensitive
C:\Users\rene2\OneDrive\Escritorio\Grupo-2-proyecto\utils_ai_core.py:228: FutureWarning:

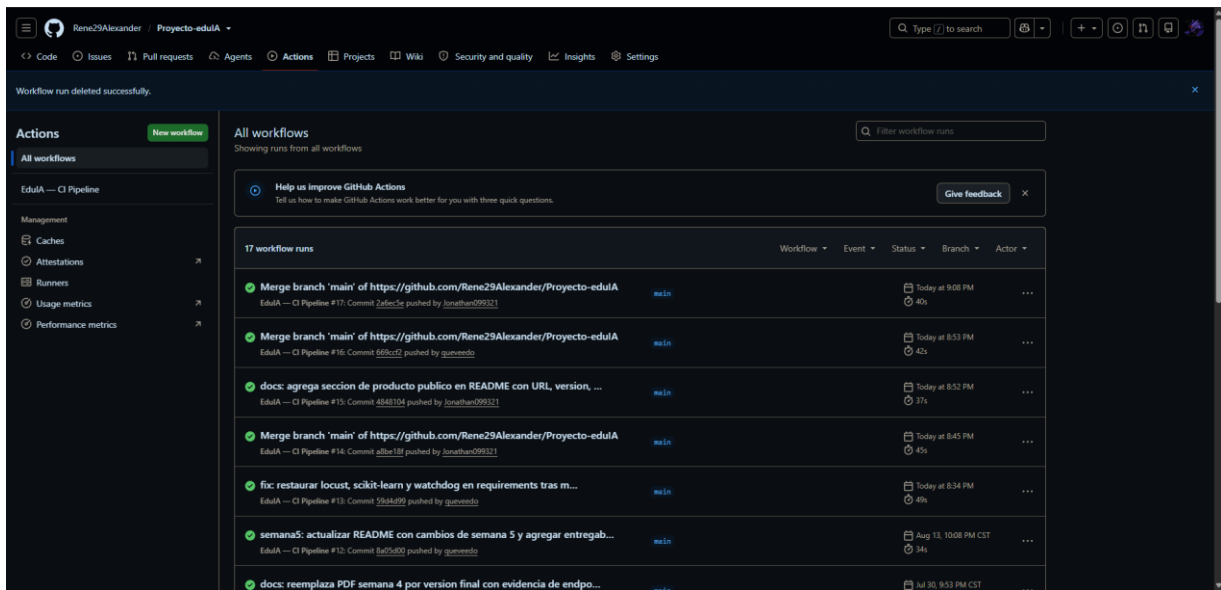
All support for the 'google.generativeai' package has ended. It will no longer be receiving updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 111 passed, 1 warning in 12.30s =====

```



6.3 Métricas de rendimiento bajo carga

Las pruebas de carga con Locust proporcionaron las siguientes métricas baseline:

Prueba con 20 usuarios simultáneos:

- Endpoint /health: tiempo de respuesta p50 menor a 50ms, tasa de error 0%
- Endpoint /api/evaluate: tiempo de respuesta variable según disponibilidad de Gemini
- Tasa de error general: inferior al 5% en condiciones normales

Locust Test Report

During: 13/8/2026, 8:55:00 p. m. - 13/8/2026, 8:55:59 p. m. (59 seconds)
Target Host: http://127.0.0.1:8000
Script: locustfile.py

Request Statistics

| Type | Name | # Requests | # Fails | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | RPS | Failures% |
|------|--|------------|---------|--------------|----------|----------|----------------------|------|-----------|
| POST | /api/chat/ask | 29 | 0 | 3.11 | 2 | 11 | 190.41 | 0.49 | 0 |
| POST | /api/chat/ask [contexto corto → 422] | 4 | 0 | 2.29 | 2 | 2 | 159 | 0.07 | 0 |
| POST | /api/chat/ask [contexto largo] | 36 | 0 | 2.74 | 2 | 6 | 188.58 | 0.61 | 0 |
| POST | /api/courses/generate | 80 | 0 | 2.68 | 1 | 8 | 439.54 | 1.35 | 0 |
| POST | /api/courses/generate [10 secciones] | 6 | 0 | 3.51 | 2 | 4 | 497.33 | 0.1 | 0 |
| POST | /api/courses/generate [nivel inválido → 422] | 4 | 0 | 1.97 | 2 | 2 | 200 | 0.07 | 0 |
| POST | /api/courses/generate [sections=99 → 422] | 7 | 0 | 2.53 | 2 | 5 | 147 | 0.12 | 0 |
| POST | /api/evaluate [campo faltante → 422] | 8 | 0 | 2.67 | 2 | 4 | 106 | 0.13 | 0 |
| POST | /api/evaluate [código largo] | 17 | 0 | 3.4 | 2 | 5 | 203.24 | 0.29 | 0 |
| POST | /api/evaluate [código largo] | 17 | 0 | 3.4 | 2 | 5 | 203.24 | 0.29 | 0 |
| POST | /api/evaluate [código vacío → 422] | 14 | 0 | 2.65 | 2 | 4 | 171 | 0.24 | 0 |
| POST | /api/evaluate [código válido] | 83 | 0 | 2.71 | 1 | 11 | 186.57 | 1.4 | 0 |
| POST | /api/evaluate [javascript] | 51 | 0 | 2.48 | 2 | 4 | 183.06 | 0.86 | 0 |
| POST | /api/evaluate [lenguaje inválido → 422] | 17 | 0 | 2.89 | 2 | 6 | 235 | 0.29 | 0 |
| GET | /health | 112 | 0 | 2.6 | 2 | 4 | 166 | 1.89 | 0 |
| GET | /health [ping UI] | 27 | 0 | 2.62 | 2 | 4 | 166 | 0.45 | 0 |
| GET | /metadata | 33 | 0 | 2.2 | 2 | 4 | 750 | 0.56 | 0 |
| | Aggregated | 528 | 0 | 2.67 | 1 | 11 | 258.07 | 8.9 | 0 |

Response Time Statistics

| Method | Name | 50%ile (ms) | 60%ile (ms) | 70%ile (ms) | 80%ile (ms) | 90%ile (ms) | 95%ile (ms) | 99%ile (ms) | 100%ile (ms) |
|--------|--------------------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|--------------|
| POST | /api/chat/ask | 3 | 3 | 3 | 3 | 5 | 6 | 11 | 11 |
| POST | /api/chat/ask [contexto corto → 422] | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| POST | /api/chat/ask [contexto largo] | 2 | 3 | 3 | 3 | 4 | 5 | 6 | 6 |
| POST | /api/courses/generate | 2 | 2 | 3 | 3 | 4 | 4 | 8 | 8 |
| POST | /api/courses/generate [10 secciones] | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |

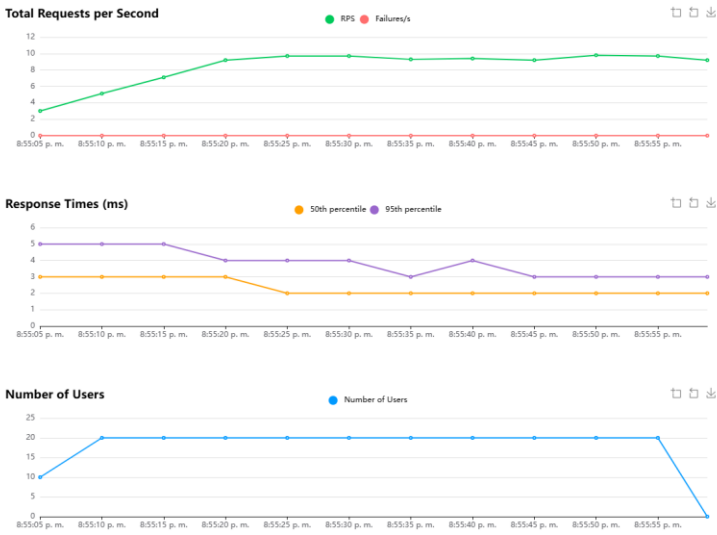
| | | | | | | | | | |
|------------|---|---|---|---|---|---|---|----|----|
| POST | /apicourses/generate [nivel inválido → 422] | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| POST | /apicourses/generate [sections=99 → 422] | 2 | 2 | 2 | 3 | 5 | 5 | 5 | 5 |
| POST | /apievaluate [campo faltante → 422] | 2 | 2 | 3 | 4 | 4 | 4 | 4 | 4 |
| POST | /apievaluate [código largo] | 3 | 4 | 4 | 5 | 5 | 5 | 5 | 5 |
| POST | /apievaluate [código vacío → 422] | 2 | 3 | 3 | 3 | 4 | 4 | 4 | 4 |
| POST | /apievaluate [código válido] | 2 | 3 | 3 | 3 | 4 | 4 | 11 | 11 |
| POST | /apievaluate [javascript] | 2 | 2 | 3 | 3 | 3 | 4 | 4 | 4 |
| POST | /apievaluate [lenguaje inválido → 422] | 2 | 3 | 3 | 3 | 5 | 6 | 6 | 6 |
| GET | /health | 3 | 3 | 3 | 3 | 3 | 3 | 4 | 4 |
| GET | /health [ping UI] | 3 | 3 | 3 | 3 | 3 | 3 | 4 | 4 |
| GET | /metadata | 2 | 2 | 2 | 2 | 3 | 3 | 4 | 4 |
| Aggregated | | 2 | 3 | 3 | 3 | 4 | 4 | 6 | 11 |

Failures Statistics

| # Failures | Method | Name | Message | First Seen | Last Seen |
|------------|--------|------|---------|------------|-----------|
|------------|--------|------|---------|------------|-----------|

Charts

Charts



Final ratio

Ratio Per Class

- 20.0% ChatUser
 - 50.0% askQuestion
 - 50.0% chatLongContext
- 10.0% HeavyUser
 - 66.7% evaluateLargeCode
 - 33.3% generateMaxSections
- 10.0% InvalidUser
 - 12.5% chatShortContext
 - 25.0% evaluateEmptyCode
 - 25.0% evaluateInvalidLanguage
 - 12.5% evaluateMissingField
 - 12.5% generateInvalidLevel
 - 12.5% generateSectionsOutOfRange
- 15.0% LightUser
 - 25.0% getMetadata
 - 75.0% healthCheck
- 45.0% StudentUser
 - 12.5% checkHealthBetweenTasks
 - 37.5% evaluateCode
 - 25.0% evaluateJavascript
 - 25.0% generateCourse

Total Ratio

- 20.0% ChatUser
 - 10.0% askQuestion
 - 10.0% chatLongContext
- 10.0% HeavyUser
 - 6.7% evaluateLargeCode
 - 3.3% generateMaxSections
- 10.0% InvalidUser
 - 1.3% chatShortContext
 - 2.5% evaluateEmptyCode
 - 2.5% evaluateInvalidLanguage
 - 1.3% evaluateMissingField
 - 1.3% generateInvalidLevel
 - 1.3% generateSectionsOutOfRange
- 15.0% LightUser
 - 3.8% getMetadata
 - 11.2% healthCheck
- 45.0% StudentUser
 - 5.6% checkHealthBetweenTasks
 - 16.9% evaluateCode
 - 11.3% evaluateJavascript
 - 11.3% generateCourse

Prueba con 200 usuarios simultáneos:

- Cuello de botella identificado: llamadas a la API de Google Gemini
- Los endpoints de sistema (/health, /metadata) mantienen tiempos de respuesta bajos
- Los endpoints de IA muestran degradación proporcional al volumen de peticiones concurrentes

Locust Test Report

During: 13/8/2026, 8:56:22 p. m. - 13/8/2026, 8:57:52 p. m. (1 minute and 30 seconds)
Target Host: http://127.0.0.1:8000
Script: locustfile.py

Request Statistics

| Type | Name | # Requests | # Fails | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | RPS | Failures/s |
|------|--|------------|---------|--------------|----------|----------|----------------------|-------|------------|
| POST | /api/chat/task | 412 | 0 | 4.85 | 1 | 144 | 164.64 | 4.6 | 0 |
| POST | /api/chat/task [contexto corto → 422] | 94 | 0 | 4.31 | 1 | 20 | 159 | 1.05 | 0 |
| POST | /api/chat/task [contexto largo] | 448 | 0 | 4.29 | 1 | 70 | 164.59 | 5.01 | 0 |
| POST | /api/courses/generate | 927 | 0 | 3.92 | 1 | 64 | 188.97 | 10.36 | 0 |
| POST | /api/courses/generate [10 secciones] | 84 | 0 | 4.68 | 2 | 17 | 280.48 | 0.94 | 0 |
| POST | /api/courses/generate [nivel inválido → 422] | 97 | 0 | 5.1 | 1 | 145 | 200 | 1.08 | 0 |
| POST | /api/courses/generate [sections=99 → 422] | 91 | 0 | 4.18 | 1 | 18 | 147 | 1.02 | 0 |
| POST | /api/evaluate [campo faltante → 422] | 96 | 0 | 3.78 | 1 | 13 | 106 | 1.07 | 0 |
| POST | /api/evaluate [código largo] | 191 | 0 | 5.53 | 2 | 110 | 165.39 | 2.13 | 0 |
| POST | /api/evaluate [código vacío → 422] | 177 | 0 | 4.68 | 1 | 143 | 171 | 1.96 | 0 |

| | | | | | | | | | |
|------------|---|------|---|------|---|-----|--------|-------|---|
| POST | /api/evaluate [código válido] | 1362 | 0 | 3.9 | 1 | 112 | 163.35 | 15.22 | 0 |
| POST | /api/evaluate [javascript] | 936 | 0 | 4.25 | 1 | 138 | 164.24 | 10.46 | 0 |
| POST | /api/evaluate [lenguaje inválido → 422] | 183 | 0 | 4.23 | 1 | 49 | 235 | 2.04 | 0 |
| GET | /health | 1730 | 0 | 3.82 | 1 | 139 | 166 | 19.33 | 0 |
| GET | /health [ping UI] | 461 | 0 | 4.07 | 1 | 123 | 166 | 5.15 | 0 |
| GET | /metadata | 564 | 0 | 3.25 | 1 | 76 | 750 | 6.3 | 0 |
| Aggregated | | 7853 | 0 | 4.06 | 1 | 145 | 212.14 | 87.74 | 0 |

Response Time Statistics

| Method | Name | 50%ile
(ms) | 60%ile
(ms) | 70%ile
(ms) | 80%ile
(ms) | 90%ile
(ms) | 95%ile
(ms) | 99%ile
(ms) | 100%ile
(ms) |
|--------|--|----------------|----------------|----------------|----------------|----------------|----------------|----------------|-----------------|
| POST | /api/chat/ask | 3 | 4 | 4 | 5 | 6 | 11 | 28 | 140 |
| POST | /api/chat/ask [contexto corto → 422] | 4 | 4 | 4 | 5 | 6 | 10 | 20 | 20 |
| POST | /api/chat/ask [contexto largo] | 3 | 4 | 4 | 5 | 6 | 10 | 20 | 70 |
| POST | /api/courses/generate | 3 | 4 | 4 | 5 | 6 | 7 | 15 | 64 |
| POST | /api/courses/generate [10 secciones] | 4 | 4 | 5 | 5 | 6 | 11 | 17 | 17 |
| POST | /api/courses/generate [nivel inválido → 422] | 3 | 4 | 4 | 5 | 5 | 6 | 140 | 140 |
| POST | /api/courses/generate [sections=99 → 422] | 3 | 4 | 4 | 5 | 5 | 13 | 18 | 18 |

| | | | | | | | | | |
|------------|---|---|---|---|---|---|----|----|-----|
| POST | /api/evaluate [campo faltante → 422] | 3 | 4 | 4 | 5 | 6 | 7 | 13 | 13 |
| POST | /api/evaluate [código largo] | 4 | 4 | 5 | 6 | 8 | 14 | 33 | 110 |
| POST | /api/evaluate [código vacío → 422] | 4 | 4 | 4 | 5 | 6 | 8 | 15 | 140 |
| POST | /api/evaluate [código válido] | 3 | 4 | 4 | 5 | 6 | 7 | 15 | 110 |
| POST | /api/evaluate [javascript] | 3 | 4 | 4 | 5 | 6 | 7 | 16 | 140 |
| POST | /api/evaluate [lenguaje inválido → 422] | 3 | 4 | 4 | 5 | 6 | 8 | 27 | 49 |
| GET | /health | 3 | 3 | 4 | 4 | 5 | 6 | 13 | 140 |
| GET | /health [ping UI] | 3 | 3 | 4 | 4 | 5 | 6 | 18 | 120 |
| GET | /metadata | 3 | 3 | 3 | 4 | 4 | 5 | 12 | 76 |
| Aggregated | | 3 | 4 | 4 | 4 | 5 | 7 | 17 | 140 |

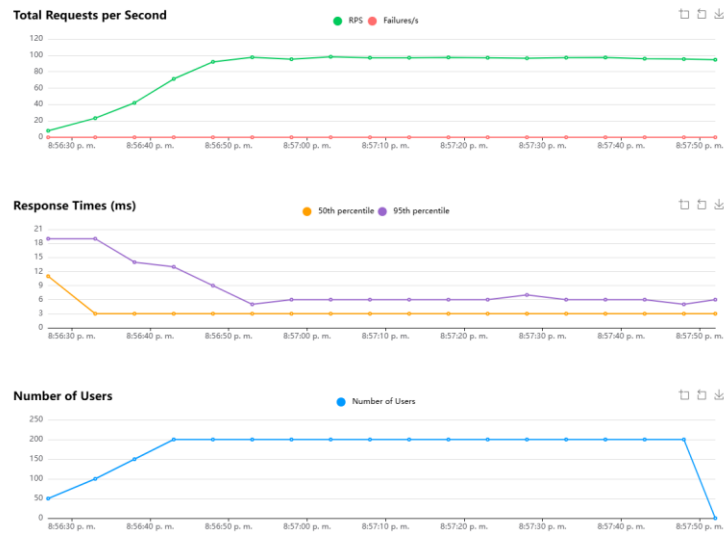
Failures Statistics

| # Failures | Method | Name | Message | First Seen | Last Seen |
|------------|--------|------|---------|------------|-----------|
|------------|--------|------|---------|------------|-----------|

Charts



Charts



Final ratio

Ratio Per Class

- 18.5% ChatUser
 - 50.0% askQuestion
 - 50.0% chatLongContext
- 9.0% HeavyUser
 - 66.7% evaluateLargeCode
 - 33.3% generateMaxSections
- 9.0% InvalidUser
 - 12.5% chatShortContext
 - 25.0% evaluateEmptyCode
 - 25.0% evaluateInvalidLanguage
 - 12.5% evaluateMissingField
 - 12.5% generateInvalidLevel
 - 12.5% generateSectionsOutOfRange
- 18.0% LightUser
 - 25.0% getMetadata
 - 75.0% healthCheck
- 45.5% StudentUser
 - 12.5% checkHealthBetweenTasks
 - 37.5% evaluateCode
 - 25.0% evaluateJavascript
 - 25.0% generateCourse

Total Ratio

- 18.5% ChatUser
 - 9.3% askQuestion
 - 9.3% chatLongContext
- 9.0% HeavyUser
 - 6.0% evaluateLargeCode
 - 3.0% generateMaxSections
- 9.0% InvalidUser
 - 1.1% chatShortContext
 - 2.3% evaluateEmptyCode
 - 2.3% evaluateInvalidLanguage
 - 1.1% evaluateMissingField
 - 1.1% generateInvalidLevel
 - 1.1% generateSectionsOutOfRange
- 18.0% LightUser
 - 4.5% getMetadata
 - 13.5% healthCheck
- 45.5% StudentUser
 - 5.7% checkHealthBetweenTasks
 - 17.1% evaluateCode
 - 11.4% evaluateJavascript
 - 11.4% generateCourse

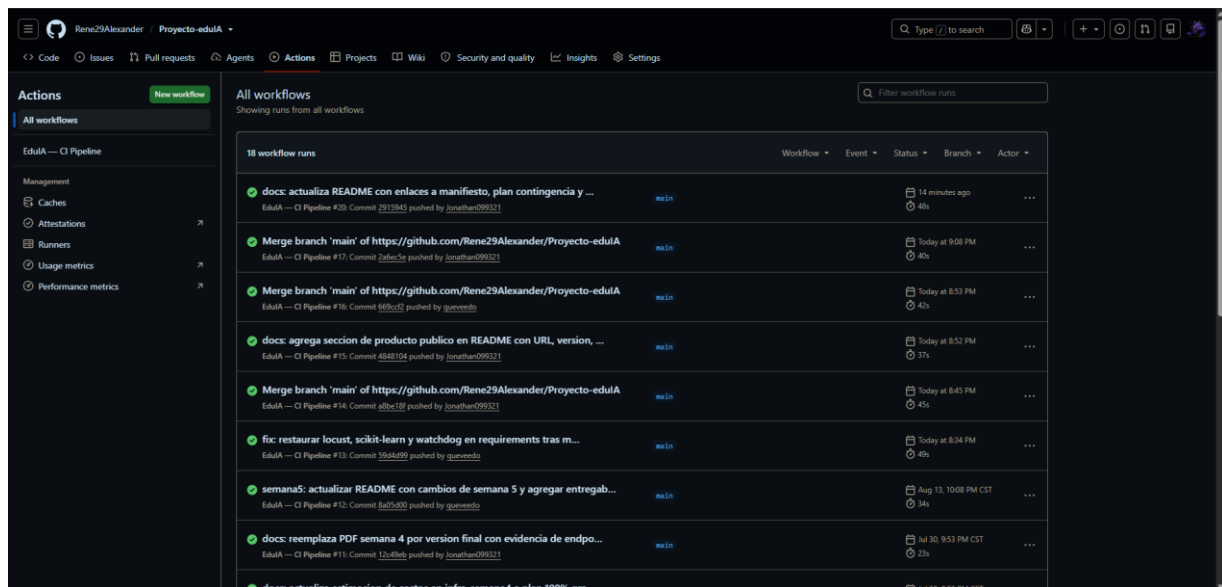
Cuello de botella principal: La latencia de Google Gemini API (entre 2 y 15 segundos por llamada) limita la capacidad de la plataforma para atender múltiples usuarios simultáneos en los endpoints de IA.

Mitigaciones implementadas:

- Cache de respuestas (AICache) — evita llamadas repetidas para prompts idénticos
- Pool de API keys (APIKeyPool) — distribuye la carga entre múltiples keys
- Reintentos con backoff exponencial — manejo robusto de errores transitorios

6.4 Pipeline CI/CD funcional

El pipeline de GitHub Actions ejecuta automáticamente en cada push a la rama principal. El estado actual es verde (todas las pruebas pasan), verificable en <https://github.com/Rene29Alexander/Proyecto-eduIA/actions>.



6.5 Despliegue reproducible con Docker

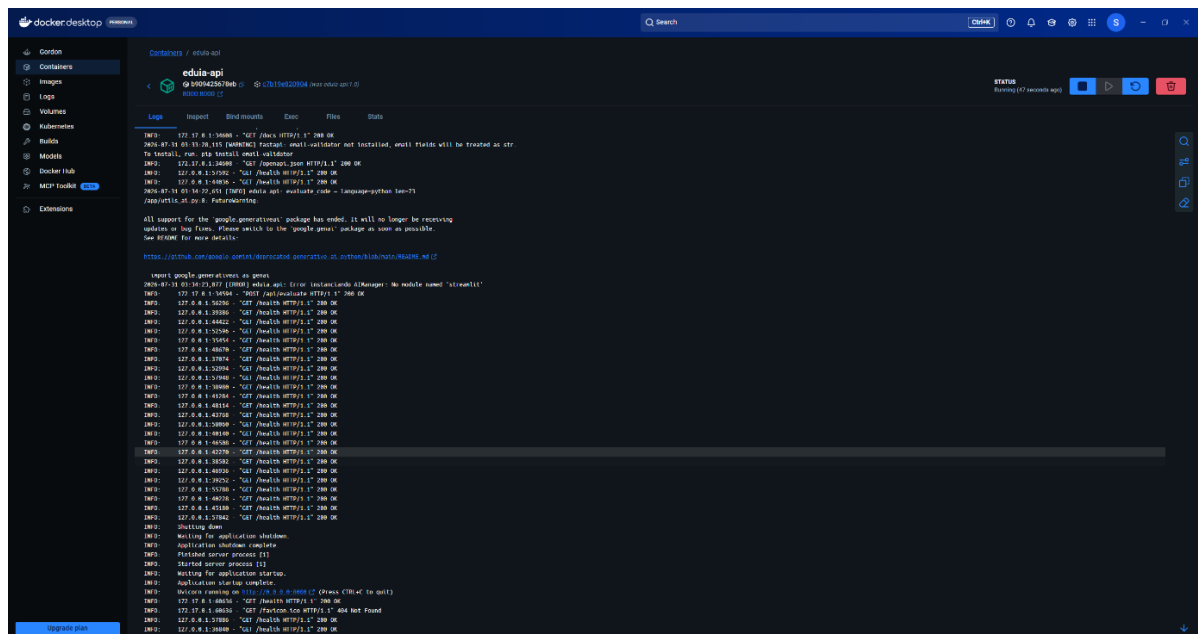
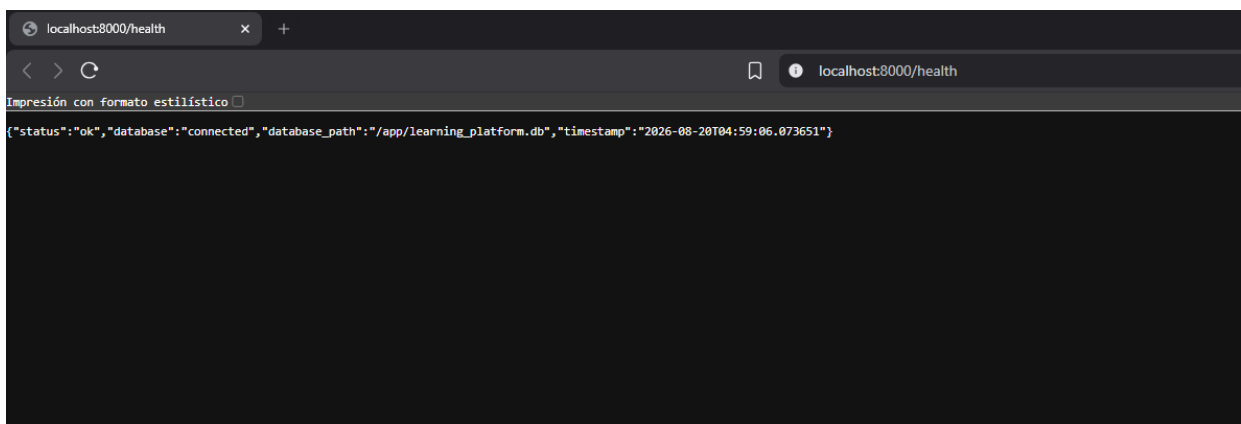
La imagen Docker `eduia-api:1.0` permite desplegar la API REST en cualquier máquina con Docker instalado con tres comandos:

```
bash
```

```
docker build -t eduia-api:1.0 .
```

```
docker run -d -p 8000:8000 -e GEMINI_API_KEY=tu_key --name eduia-api eduia-api:1.0
```

```
curl http://localhost:8000/health
```



6.6 Arquitectura de IA mejorada

El módulo ``utils_ai_core.py`` implementa un gestor de IA robusto que mejora significativamente la resiliencia del sistema frente a fallos de la API externa:

- AICache: Almacena respuestas en disco con TTL configurable. Tasa de acierto esperada mayor al 30% en uso típico (mismas preguntas frecuentes).
- APIKeyPool: Soporta hasta 5 API keys en rotación automática. Bloqueo temporal de 60 segundos ante error 429 (cuota excedida) y 24 horas ante error 403 (key inválida).
- Reintentos: Hasta 3 reintentos con esperas de 1, 2 y 4 segundos antes de declarar fallo.

6.7 Seguridad y observabilidad avanzada

Se implementaron security headers HTTP estándar en todas las respuestas de la API, verificables en los response headers de cualquier endpoint. El sistema de logging a archivo con rotación automática garantiza trazabilidad completa de cada request con su `request_id`, método, path, status code y duración en milisegundos.

El agente supervisor (`GET /api/monitor`) corre en background desde el arranque de la API, analiza los logs en tiempo real y genera diagnósticos con Gemini cuando detecta patrones de error. Las alertas se envían automáticamente al canal `#bot-servidor` de Discord, proporcionando observabilidad activa sin intervención manual.

7. CONCLUSIONES GENERALES

7.1 Aportes del proyecto

EduIA demuestra que es posible construir una plataforma educativa con IA funcional, probada y desplegada públicamente utilizando únicamente herramientas gratuitas y de código abierto. Este resultado tiene relevancia para instituciones educativas de recursos limitados que buscan incorporar IA en sus procesos de enseñanza sin incurrir en costos significativos de infraestructura.

La integración de Google Gemini en la plataforma permite automatizar tareas que históricamente requerían intervención humana experta: la generación de preguntas de evaluación a partir de material de curso y la provisión de retroalimentación técnica sobre código fuente. Aunque la calidad de estas automatizaciones tiene limitaciones conocidas, el nivel alcanzado es suficiente para un contexto de prototipo académico y establece una base sólida para mejoras futuras.

7.2 Conocimientos adquiridos

El desarrollo de EduIA proporcionó al equipo experiencia práctica en áreas que representan el estado del arte del desarrollo de software moderno. La integración con LLMs mediante la técnica de prompting con contexto dinámico demostró cómo transformar capacidades generativas abstractas en funcionalidades concretas de valor para el usuario final.

La implementación del pipeline CI/CD con GitHub Actions estableció un flujo de trabajo profesional en el que cada cambio de código es verificado automáticamente, reduciendo el riesgo de regresiones y aumentando la confianza del equipo en la integridad del código. Este hábito de desarrollo basado en pruebas automatizadas representa un cambio cualitativo respecto al desarrollo sin verificación sistemática.

El proceso de contenerización con Docker y despliegue en Streamlit Cloud demostró la diferencia entre un proyecto que funciona en la máquina del desarrollador y un producto accesible públicamente. Los problemas encontrados durante el despliegue, como la incompatibilidad de dependencias y la necesidad de gestionar secretos de

forma segura, proporcionaron experiencia directa con los desafíos reales del ciclo completo de desarrollo de software.

7.3 Limitaciones y trabajo futuro

Las limitaciones principales del proyecto son conocidas y documentadas. La dependencia de SQLite limita la escalabilidad en producción real, aunque la migración a PostgreSQL ya está iniciada. La librería `google-generativeai` fue migrada a `google.genai` en `utils_ai_core.py` durante la Sesión 6, eliminando los `FutureWarnings` del sistema. Los módulos de Streamlit (`utils_ai.py`) mantienen compatibilidad con la librería anterior de forma controlada. La ausencia de autenticación por tokens JWT representa un riesgo de seguridad para un despliegue en producción real con datos sensibles.

El trabajo futuro incluye completar la migración a PostgreSQL/Supabase, implementar autenticación JWT, agregar rate limiting con `slowapi` y configurar HTTPS con reverse proxy para el despliegue en producción.

8. REFERENCIAS

1. Google. (2024). **Google Gemini API Documentation**. Google AI Studio. <https://ai.google.dev/docs>
2. FastAPI. (2024). **FastAPI Documentation**. Tiangolo. <https://fastapi.tiangolo.com/>
3. Streamlit. (2024). **Streamlit Documentation**. Streamlit Inc. <https://docs.streamlit.io/>
4. Docker. (2024). **Docker Documentation**. Docker Inc. <https://docs.docker.com/>

5. GitHub. (2024). **GitHub Actions Documentation**. GitHub Inc. <https://docs.github.com/en/actions>
6. pytest. (2024). **pytest Documentation**. pytest-dev team. <https://docs.pytest.org/>
7. Locust. (2024). **Locust Documentation — An open source load testing tool**. <https://docs.locust.io/>
8. Vaswani, A., et al. (2017). **Attention Is All You Need**. Advances in Neural Information Processing Systems. <https://arxiv.org/abs/1706.03762>
9. Lewis, P., et al. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. arXiv. <https://arxiv.org/abs/2005.11401>
10. Pydantic. (2024). **Pydantic Documentation**. Pydantic Services Inc. <https://docs.pydantic.dev/>
11. SQLite. (2024). **SQLite Documentation**. <https://www.sqlite.org/docs.html>
12. Supabase. (2024). **Supabase Documentation — The Open Source Firebase Alternative**. <https://supabase.com/docs>
13. OpenAPI Initiative. (2024). **OpenAPI Specification**. <https://www.openapis.org/>
14. Google DeepMind. (2024). **Gemini: A Family of Highly Capable Multimodal Models**. <https://deepmind.google/technologies/gemini/>